

grlc: a tool for interactive and incremental reactive synthesis

Scott C. Livingston

January 21, 2024

Abstract

grlc is an open source tool suite for GR(1) synthesis and related activities. Development is motivated by current research in formal methods for robotics, including motion planning with temporal logic specifications. It has several novel features beyond a fast implementation of basic GR(1) synthesis. Users can interact with binary decision diagrams (BDDs) and make live queries to intermediate values of a μ -calculus fixed-point computation, which in turn provide the basis for strategy automaton construction. Another novel feature is the ability to load an existing strategy and modify parts of it following changes to the original specification. Besides visual formats intended for human review, grrlc can produce output in standard formats, including JSON, that enable integration with other tools. To support verifying results, output can be in Promela, which allows model checking by Spin.

1 Introduction

The primary distinction of formal synthesis from other problems in control theory is the specification of desired behavior in a formal language such as linear temporal logic (LTL). In this work, we are concerned with reactive synthesis for the fragment of LTL called generalized reactivity rank 1, or GR(1) [7]. Specifications in GR(1) have solutions that produce trajectories of infinite duration and that are defined by discrete steps taken in turn with an adversarial environment. Intuitively, the problem is to find a strategy that guarantees a set of goals will be reached repeatedly, assuming that the environment repeatedly satisfies a set of its own goals. Precisely, the specification is

$$\bigwedge_{j=1}^m \Box \Diamond \psi_j^{\text{env}} \implies \bigwedge_{i=1}^n \Box \Diamond \psi_i^{\text{sys}}, \quad (1)$$

where ψ_j^{env} are the environment goals (i.e., sets of states that the adversary must repeatedly reach), and ψ_i^{sys} are the system goals (i.e., sets of states that the constructed strategy must reach). A game is defined by initial states and

feasible transitions among states with these goals. The left and right side of the implication in (1) may be referred to as liveness requirements, and the feasible transitions may be referred to as safety requirements. Note that some authors encode the initial states and feasible transitions in LTL and add them to the formula (1).

Despite its apparent simplicity, many different kinds of LTL formulae can be rewritten in the GR(1) template. Application to more general dynamical systems (e.g., having dense time) is achieved by constructing discrete abstractions that model the original system in a way where synthesis can be applied, e.g., through bisimulation [1].

In this paper, we provide a brief introduction to **gr1c**, a tool suite for GR(1) synthesis and related activities. The source code is available at <https://github.com/tulip-control/gr1c>

2 Related work

gr1c was first released in the year 2012. Before it, the first implementation of GR(1) known to the authors was in Java with JTLV [11], [12]. This tool had the advantage of being easy to distribute because it required only an appropriate version of the Java VM installed. However, it was released as an opaque JAR file and eventually not maintained. For research that requires modifying the basic fixed-point process, something new was required. Thus motivated, and to overcome performance and usability concerns with that first implementation, **gr1c** was created.

Since the beginning, **gr1c** has been developed to support integration with other software, in particular the Temporal Logic Planning toolbox, Tulip [4]. Tulip is written in Python and supports expressing and solving formal synthesis problems with hybrid systems. Internally, Tulip passes part of a problem to GR(1) solvers, such as **gr1c**, and incorporates the resulting strategy into the solution of the original problem. More recently, the default solver in Tulip is **omega**, a Python package that can do GR(1) synthesis and more [5]. Documentation is available at <http://tulip-control.org>

Perhaps the most similar tool is Slugs [3]. Unlike **gr1c**, which is written in standard C and emphasizes performance, Slugs is written in C++ and prioritizes easy extension. The Slugs distribution includes plugins that implement algorithms from the authors' research, such as adding costs or producing reports.

3 Basic aspects

3.1 Summary of operation

Solving (1) essentially involves two steps:

1. compute the winning set, i.e., the set of states from which there is some strategy that guarantees satisfaction of the specification;

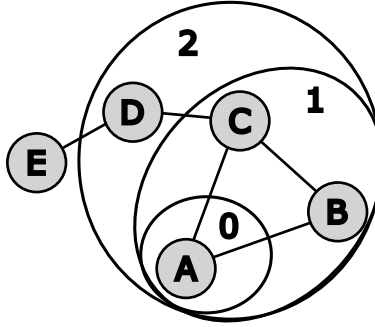


Figure 1: Illustration of the sets of states (sublevel sets) found during winning set computation. State A represents a system goal, and the edges between vertices represent safe feasible transitions. The initial set (0) includes only state A. After two iterations, states B and C are in set 1, and state D is in set 2.

2. from the intermediate values of the winning set computation, enumerate a strategy (as a finite graph) that guarantees system safety and liveness.

Completing the first step and checking whether initial states are contained in the winning set, without constructing an explicit strategy, is known as *checking realizability*. Most of the computation is performed in terms of sets of states using binary decision diagrams (BDDs) [2]. The winning set is found by iterating a μ -calculus formula until a fixed point is reached [8]. Currently, the only supported external package of BDD operations is CUDD (CU Decision Diagram) [13]. Work is in progress to support other BDD packages.

The sets created during fixed-point iterations are known as *sublevel sets*. With these, a strategy is constructed by successively moving into inner subsets until a goal state is reached and, then, repeating this for the next system goal. Note that the environment may take different actions at each state, so the (system) strategy must have an action to take in response for each. Moving into the next sublevel set is not necessarily possible at a given time, but because of the environment liveness assumption, eventually it will be. This setting is illustrated in Figure 1.

3.2 Syntax example

Problems are specified in a plaintext file format. Variables are declared as Boolean or as having domains of ranges of integers. LTL operators $\Box$, $\Diamond$, and $\bigcirc$ are written as `[]`, `<>`, and `'` (primed variable), respectively. Initial constraints, transition rules, and goals are written in separate blocks with names `ENVINIT`, etc.

For example, the following specification defines two variables, where `x` is uncontrolled and is assumed to repeatedly toggle `True` and `False`. The synthesis problem is to repeatedly have `y` be `True` when `x` is, while also repeatedly have `y` be `False`.

```

# Example 1
ENV: x;
SYS: y;

ENVINIT: !x;
ENVTRANS: [] (x <-> !x');
ENVGOAL: [] <>x;

SYSINIT: y;
SYSGOAL:
  [] <>(y&x)
& [] <>(!y);

```

As an example of variables with integer values, consider the following specification. The variable `y` must be repeatedly assigned 0 and 2, but at each time step, it can only change by ± 1 .

```

# Example 2
SYS: y [0,2];

SYSINIT: y=0;

SYSTRANS:
  [] ((y=2) -> (y'=1 | y'=2))
& [] ((y=1) -> (y'=0 | y'=1 | y'=2))
& [] ((y=0) -> (y'=0 | y'=1));

SYSGOAL: [] <>y=0 & [] <>y=2;

```

3.3 Representing variables of other types as atomic propositions

Binary decision diagrams typically have Boolean valued variables, also known as atomic propositions, that are either `True` or `False`. `gr1c` thus handles integer variables by encoding them as bitvectors, i.e., ordered sets of atomic propositions, as follows. Let x be an integer variable with domain $\{0, 1, \dots, n\}$, where $n \geq 1$. Let

$$k = \lceil \log_2(n) \rceil$$

and create k atomic propositions $x_0^{\text{bit}}, x_1^{\text{bit}}, \dots, x_{k-1}^{\text{bit}}$. As the names suggest, these are interpreted as bits encoding the value of x . E.g., $x = 5$ (supposing that $n \geq 5$) would be represented by the state $\{x_0^{\text{bit}}, x_2^{\text{bit}}\}$. Since n might not be a power of 2, several states need to be precluded in order for the domain of x to be correctly represented. In GR(1) formulae, this is achieved by adding restrictive subformulae if $n < k$,

$$\Box (\neg \chi_{x=n+1} \wedge \neg \chi_{x=n+2} \wedge \dots \wedge \neg \chi_{x=k}),$$

where $\chi_{x=k}$ is the characteristic formula for the state of $x = k$, which is actually a conjunction of the atomic propositions $x_0^{\text{bit}}, x_1^{\text{bit}}, \dots, x_{k-1}^{\text{bit}}$, some negated; e.g., if $k = 3$,

$$\chi_{x=5} = x_0^{\text{bit}} \wedge \neg x_1^{\text{bit}} \wedge x_2^{\text{bit}}.$$

4 Interactivity

If started with the command-line flag `-i`, **gr1c** will compute the sublevel sets from the fixpoint computation of the winning set and then start a prompt for interactively probing the data structures, modifying the specification, and finding feasible environment and system actions from any given state.

For example, the following session uses the first specification above.

```
>>> var
x (0), y (1)
>>> numgoals
2
>>> printgoal 0
(y&x)
>>> printgoal 1
(!y)
>>> sysnext 1 1 1 1
0
---
```

The first four commands are self-explanatory. The final command, **sysnext**, gets all feasible next system states that make progress (in terms of winning sets) toward the goal `!y` from state `1 1` (`x` and `y` both `True`). Other commands are available to investigate intermediate values (as outlined in Section 3.1 and Figure 1). Enter **help** to get a summary of all available commands.

A Python wrapper for interactive sessions is included in Tulip. Finally, there is optional linking with GNU Readline for an enhanced prompt with command history, etc.

5 Incremental synthesis

The algorithms presented in [8], [9], and [10] are provided through the application-programming interface (API) of **gr1c** and via the command-line program **gr1c patch**. Specifically, goal appending can be achieved using

```
gr1c patch -f PHI [...]
```

where **PHI** is an arbitrary state formula (i.e., without temporal operators), which is parsed using the same syntax as for GR(1) specifications. Also, the switch `-m` can be included in the above command to indicate that some variables are integer-valued and can be included in the computation of a norm (regarding

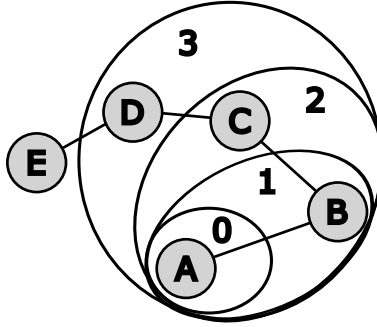


Figure 2: After modifying Figure 1 by removing the edge between states A and C, new sublevel sets are found. In *gr1c*, this modification to an existing specification and strategy is expressed by a **restrict** command.

the variables as elements in a vector), from which the Dist function is obtained; otherwise Dist is taken to be constant. Goal removal can be achieved using `gr1c patch -r i`, where *i* is the index, corresponding to removal of the system liveness (goal) subformula ψ_i^{sys} .

The expression of changes to reachability is somewhat more complicated than changes to liveness. This kind of change is illustrated in Figure 2. The approach taken in *gr1c* is to define a separate file type known as an “edge set change file.” These files consist of two parts: first a list of states considered to be in the neighborhood *N*; and second, a sequence of **restrict**, **relax**, or **blocksys** commands that remove or add edges from the corresponding game graph (equivalently, add or remove feasible transitions to the specification). As in other files for working with *gr1c* and following the convention of Python and UNIX shell scripts, comments begin with **#**. For example, consider a specification with two atomic propositions (also known as Boolean variables) *p* and *q*, and suppose that the neighborhood *N* is $\{\emptyset, \{q\}, \{p, q\}\}$, i.e., states that satisfy $(\neg p \wedge \neg q) \vee (\neg p \wedge q) \vee (p \wedge q)$. To change the game graph by removing the controlled edge from $\emptyset$ to $\{q\}$, the file is

```
0 0
0 1
1 1
restrict 0 0 0 1
```

6 Model checking

While algorithms for formal synthesis produce controllers that are “correct-by-construction,” in practice there can be errors in the implementations of solvers that result in controllers that are not correct. These errors can go unnoticed because nontrivial specifications have complex solutions that are difficult to inspect. This practical issue is especially relevant in research, where new code

is written but often not subject to rigorous review, in contrast to algorithms written in papers. To protect against this, `gr1c` can express strategies in the Promela language of Spin [6], thereby facilitating model checking.

7 Conclusion

Now more than 11 years old, development and growth of `gr1c` is ongoing. Most uses of `gr1c` have been in research. Future work will also be in practical applications by leveraging the generality, speed, and quality of its implementation.

Acknowledgements

This work was partially supported by United Technologies Corporation and IBM, through the industrial cyberphysical systems (iCyPhy) consortium. The author thanks Richard M. Murray and Ioannis Filippidis for feedback and discussion since the early days of `gr1c`.

References

- [1] R. Alur, T. A. Henzinger, G. Lafferriere, and G. J. Pappas. Discrete abstractions of hybrid systems. *Proceedings of the IEEE*, 88(7):971–984, July 2000.
- [2] R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.
- [3] R. Ehlers and V. Raman. Slugs: Extensible GR(1) synthesis. In *28th International Conference on Computer Aided Verification (CAV)*, pages 333–339, 2016.
- [4] I. Filippidis, S. Dathathri, S. C. Livingston, N. Ozay, and R. M. Murray. Control design for hybrid systems with TuLiP: The Temporal Logic Planning toolbox. In *Proc. of IEEE Conference on Control Applications (CCA)*, pages 1030–1041, 2016.
- [5] I. Filippidis and R. M. Murray. Layering assume-guarantee contracts for hierarchical system design. *Proceedings of the IEEE*, 106(9):1616–1654, 2018.
- [6] G. J. Holzmann. The model checker Spin. *IEEE Transactions on Software Engineering*, 23(5):279–295, May 1997.
- [7] Y. Kesten, N. Piterman, and A. Pnueli. Bridging the gap between fair simulation and trace inclusion. *Information and Computation*, 200:35–61, 2005.

- [8] S. C. Livingston. *Incremental Control Synthesis for Robotics in the Presence of Temporal Logic Specifications*. PhD thesis, California Institute of Technology, Pasadena, California, USA, 2015.
- [9] S. C. Livingston and R. M. Murray. Hot-swapping robot task goals in reactive formal synthesis. Technical report, California Institute of Technology, September 2014. <http://resolver.caltech.edu/CaltechCDSTR:2014.001>.
- [10] S. C. Livingston, P. Prabhakar, A. B. Jose, and R. M. Murray. Patching task-level robot controllers based on a local μ -calculus formula. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 4573–4580, Karlsruhe, Germany, May 2013.
- [11] A. Pnueli, Y. Sa’ar, and L. D. Zuck. Jtlv: A framework for developing verification algorithms. In *International Conference on Computer Aided Verification (CAV)*, pages 171–174, 2010.
- [12] Y. Sa’ar. JTLV project home site. <https://web.archive.org/web/20120225063529/http://jtlv.ysaar.net/>. Accessed: 2012-02-25.
- [13] F. Somenzi. CUDD: CU decision diagram package. <https://web.archive.org/web/20150215010018/http://vlsi.colorado.edu/fabio/CUDD/cuddIntro.html>. Accessed: 2015-02-15.